

Improving Semantic Edge Detection using Realistic Indoor Synthetic Data

Michelle Appel
Donders Institute
Nijmegen, the Netherlands
michelle.appel@ru.nl

Second Author
Institution2
First line of institution2 address
secondauthor@i2.org

Abstract

In this paper, we present a novel approach to edge detection by training our model on synthetic interior images. We demonstrate that our model outperforms state-of-the-art methods on real-world datasets such as NYUDv2 and BSDS500, with particularly crisp and accurate edges. Not only does this approach improve performance, it also allows for greater control and diversity in the training data. In a nutshell, our synthetic interior dataset and trained model produce edges that are not only more accurate, but also "crispier" and more visually pleasing.

1. Introduction

In the ever-evolving field of computer vision, semantic edge detection is a crucial component in various computer vision tasks. It refers to the process of identifying and outlining the edges of objects within an image, where these edges are understood in terms of their semantic meaning. This means that instead of merely detecting changes in color or intensity, semantic edge detection aims to understand the edges as part of distinct objects or categories within the image. This approach is more advanced than traditional edge detection, as it requires the system to have some understanding of the objects in the image and their context.

Traditional methods like the Canny and Sobel detectors have been cornerstone approaches for edge detection [1, 3]. However, these techniques, primarily based on gradient operations, often fall short in complex visual environments with varied textures or lighting conditions.

The advent of deep learning, especially Convolutional Neural Networks (CNNs), has significantly reshaped edge detection [4]. CNNs effectively capture a wide range of image features, enhancing the accuracy and reliability of edge detection. Despite this progress, a fundamental challenge persists: the scarcity of diverse, high-quality training data. The conventional reliance on manually annotated datasets not only introduces potential errors and biases but also limits the variety of training datasets. This limitation is partic-

ularly problematic in specialized areas such as visual prosthetics, where accurate edge detection is vital for creating meaningful visual representations, like phosphenes, for the visually impaired [2].

Our research proposes a novel methodology that utilizes synthetic data – computer-generated images that closely resemble real-world scenarios. We harness the Unity engine, known for its robust 3D rendering capabilities, to create intricate indoor scenes, providing a rich source of synthetic training data complete with precise edge labels [5].

Yet, synthetic data often lacks the subtle imperfections found in real-world imagery. To address this, we employ a domain adaptation technique, specifically CycleGAN, to enhance the realism of our synthetic images, a process known as domain transfer [7]. This ensures that our model, trained on enriched synthetic data, is adept at processing actual photographic images.

Our approach not only marks a significant advancement in training robust edge detection models but also circumvents the limitations associated with traditional data collection and annotation methods. Importantly, it holds substantial promise for applications in visual prosthetics, offering a potential pathway to improve phosphene vision for the visually impaired. In the subsequent sections, we explore our methodology in detail, present our experimental findings, and discuss the wider implications of our work in the expansive field of computer vision.

2. Related Work

The field of edge detection, a fundamental aspect of image processing and computer vision, has evolved significantly over the years, leading to a variety of approaches and methodologies. In this section, we explore the key developments in edge detection, focusing on traditional methods, advancements through deep learning, the role of synthetic data, and domain adaptation techniques.

2.1. Traditional Edge Detection Methods

The foundation of edge detection in digital imagery was laid by pioneering techniques such as the Canny edge detec-

tor [1] and the Sobel operator [3]. These methods, primarily gradient-based, were designed to identify edges through changes in intensity across an image. Canny’s method, in particular, became a standard in the field due to its balance between edge detection and noise reduction. However, these traditional approaches often struggled with complex real-world imagery, limited by their reliance on low-level features and inability to adapt to varied textures and lighting conditions.

2.2. Deep Learning in Edge Detection

The emergence of deep learning, especially Convolutional Neural Networks (CNNs), brought a paradigm shift in edge detection techniques [4]. CNNs, with their ability to learn hierarchical feature representations, have demonstrated superior performance in detecting edges compared to traditional methods. A key advancement in this area was the development of the Bi-Directional Cascade Network (BDCN) by He et al. [2], which utilized a cascade structure to integrate multi-scale and multi-level information for edge detection. This approach allowed for the capture of both low-level details and high-level semantic information, resulting in more accurate and robust edge detection.

2.3. Synthetic Data in Computer Vision

The use of synthetic data has been explored as a means to overcome the limitations of real-world dataset collection and annotation. Synthetic datasets, generated through computer graphics techniques, offer a controlled environment to create a wide range of scenarios with accurate annotations. Ros et al. [5] demonstrated the effectiveness of synthetic data in semantic segmentation, showing that models trained on synthetic datasets could generalize to real-world images. This revelation paved the way for exploring synthetic data in edge detection, where the need for diverse and accurately labeled data is paramount.

2.4. Domain Adaptation Techniques

A significant challenge with synthetic data is the domain gap between synthetic and real-world imagery. Domain adaptation techniques, particularly those using Generative Adversarial Networks (GANs), have been employed to bridge this gap. CycleGAN, introduced by Zhu et al. [7], became a notable tool in this area, enabling the transformation of images from one domain to another without paired examples. This technique is instrumental in enhancing the realism of synthetic data, making it more suitable for training models that perform well on real-world images.

In summary, the related work in edge detection highlights a trajectory from traditional gradient-based methods to sophisticated deep learning models that leverage the strengths of synthetic data and domain adaptation. Our research builds upon these foundations, proposing a novel ap-

proach that combines the generation of detailed synthetic data with advanced domain transfer techniques to train edge detection models that are not only accurate but also versatile in their application.

3. Methods

This section delves into the structured methodology of our research, emphasizing the generation of synthetic data, its assimilation into the deep learning framework, advanced domain adaptation, and the detailed architecture of our edge detection model. We also introduce our Unity module, designed for synthetic data generation, which is available on GitHub for community use.

3.1. Dynamic Synthetic Data Generation in Unity

Utilizing Unity’s powerful rendering capabilities, we developed a module for the dynamic generation of synthetic datasets. This module, available on GitHub, programmatically creates intricate indoor scenes, essential for diverse and realistic training data.

3.1.1 Scene Composition

Leveraging the ArchVizPro assets [6], scenes are constructed to replicate a wide array of indoor environments, rich in detail and complexity. This approach ensures a broad spectrum of textures, lighting, and spatial layouts.

3.1.2 Probabilistic Camera Setup

To realistically simulate human vision and various camera angles, we implemented a stochastic camera model in Unity. This model captures scenes from diverse perspectives, crucial for generating a synthetic dataset that closely mirrors real-world scenarios.

- **Ground Position** (x, z): We uniformly sample camera positions within each scene’s navigable area. This sampling method ensures that our camera viewpoints are varied and representative of realistic human perspectives in different spatial contexts.
- **Height** (y): Camera heights are sampled from a normal distribution $N(\mu_y, \sigma_y^2)$. This mimics the natural variation in the eye-level of different individuals or the mounting height of cameras, thus introducing vertical diversity in our visual data.
- **Orientation**: We sample orientation angles to introduce naturalistic camera movements:
 - **Yaw** (θ_y): Sampled from a uniform distribution $U(0, 360)$, yaw represents horizontal rotation. This simulates turning the head or camera

left and right, offering a panoramic view of the environment.

- **Pitch** (θ_x): Following a normal distribution $N(0, \sigma_{\theta_x}^2)$, pitch denotes vertical rotation. It reflects the up and down tilting of the head or camera, as in nodding, which is vital for capturing the vertical extent of a scene.
- **Roll** (θ_z): Sampled from $N(0, \sigma_{\theta_z}^2)$, roll indicates the camera’s tilt, akin to a head’s tilt to the side. This subtle rotation adds to the realism by imitating how humans or handheld cameras might slightly tilt while capturing an image.
- **Field of View (FOV)**: The FOV, sampled from $N(\mu_{FOV}, \sigma_{FOV}^2)$, controls the extent of the observable world at any given moment. Variability in FOV allows us to simulate different lens zoom levels, ranging from wide-angle (capturing more of the scene) to telephoto perspectives (focused on a smaller area), thus adding another layer of realism to our synthetic dataset.

3.2. Custom Shader for Edge Detection

In our approach, we employ the Roberts Cross operator within our custom shader to achieve high-precision edge detection, characterized by lines of one pixel in width. This operator is particularly effective for outlining the boundaries of objects with remarkable accuracy, which is crucial for our edge detection objectives.

The Roberts Cross operator functions on a simple yet efficient principle. It uses a pair of 2x2 convolution kernels to compute the gradient of an image, capturing the spatial intensity change at each pixel. These kernels are defined as follows:

$$G_x = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

$$G_y = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}$$

The application of these kernels is straightforward:

- **Gradient Calculation**: For each pixel, the gradient components in the x and y directions are calculated by convolving these kernels with the instance map.
- **Edge Magnitude**: The magnitude of the gradient at each pixel, indicative of the edge strength, is then calculated using the formula:

$$\text{Magnitude} = \sqrt{G_x^2 + G_y^2}$$

This value is a measure of the sharpness of the edge at that pixel.

- **Edge Orientation**: Additionally, the orientation of the edge is determined by calculating the arctangent of the ratio of the y-gradient to the x-gradient. This helps in understanding the direction of the edge.

By incorporating this operator into our shader, we significantly enhance the definition of object boundaries within images. This approach, when synergistically combined with instance segmentation, enables our model to detect and delineate the contours of individual objects with exceptional precision. Consequently, this integration leads to a highly accurate and detailed representation of object boundaries, a critical factor in elevating the efficacy and accuracy of our edge detection system.

3.3. Domain Transfer with CycleGANs

To effectively bridge the gap between synthetic and real-world imagery, we integrate a domain transfer mechanism using Cycle-Consistent Generative Adversarial Networks (CycleGANs). This adaptation is crucial in our edge detection system, as it significantly enhances the model’s ability to generalize from controlled, synthetic environments to varied, real-world settings.

3.3.1 Conceptual Overview

CycleGANs excel in translating images from one domain to another, say from synthetic to real-world, without requiring paired examples. This feature is particularly beneficial in scenarios where exact correspondences between synthetic and real images are unfeasible. For instance, while our simulated images appear visually realistic, they might differ from real images in aspects like color distribution, texture, or noise patterns. CycleGANs address this by:

- **GAN Framework**: Employing two networks — a generator and a discriminator. The generator aims to transform synthetic images to resemble real-world ones, while the discriminator assesses the authenticity of these transformed images.
- **Cycle Consistency**: Ensuring that key attributes of original images are preserved during translation. This is achieved through a cycle consistency loss, which penalizes significant deviations from the original image content.

3.3.2 Mathematical Formulation

CycleGAN typically involves two mappings: $G : X \rightarrow Y$ and $F : Y \rightarrow X$, facilitating transformations between the synthetic domain (X) and the real-world domain (Y). In our research, we exclusively utilize G to perform transformations from the synthetic to the real domain. Specifically,

we focus on mapping synthetic images to their corresponding real-world counterparts. The adversarial loss, which encourages realistic transformations, is defined as follows:

$$\mathcal{L}_{GAN}(G, D_Y, X, Y) = \mathbb{E}_{y \sim p_{data}(y)} [\log D_Y(y)] + \mathbb{E}_{x \sim p_{data}(x)} [\log(1 - D_Y(G(x)))]$$

Additionally, we employ the cycle consistency loss to preserve content integrity, which is defined as:

$$\mathcal{L}_{cyc}(G, F) = \mathbb{E}_{x \sim p_{data}(x)} [\|F(G(x)) - x\|_1] + \mathbb{E}_{y \sim p_{data}(y)} [\|G(F(y)) - y\|_1]$$

3.3.3 Application and Significance

Domain transfer is instrumental in our approach, transforming synthetic images to mimic the nuances of real-world images. Without it, synthetic images, despite their realistic appearance, might fail to capture the subtle variances found in real-world settings, leading to poor model generalization. CycleGAN’s ability to perform unpaired image-to-image translation is particularly valuable, allowing us to train on diverse, non-corresponding datasets. The final output of this process is a set of transformed images, which, while retaining their essential structure, exhibit the textural and lighting characteristics of real-world scenes. We utilize the generator from the synthetic to real domain, enabling our edge detection model to be trained on these adapted images. This training significantly enhances the model’s performance and applicability in real-world scenarios, making it a vital component of our edge detection framework.

3.4. Model Architecture and Training

Our choice of the edge detection model is a testament to our commitment to achieving state-of-the-art results. We have opted for a modified Bi-Directional Cascade Network (BDCN) [2] coupled with a VGG-16 backbone. This combination is renowned for its superior performance in edge detection tasks, making it an ideal fit for our rigorous research needs.

VGG-16 Backbone: The VGG-16 model, with its 13 convolutional layers using 3×3 filters followed by three fully connected layers, excels in extracting intricate image features. Its architecture, characterized by small filter sizes and deeper layers, allows for capturing a wide range of features from simple to complex, a crucial factor in accurately identifying edges in diverse and challenging visual environments.

Cascade Structure: The BDCN framework is specially designed for edge detection. Its cascading architecture integrates side-output layers with bi-directional connections, enabling the model to effectively combine multi-scale and

multi-level feature information. This approach allows for capturing both fine details and broader context, resulting in more accurate and robust edge detection. The bi-directional flow of information in BDCN helps in refining the edge predictions, making it adept at handling a variety of edge types and complexities.

Training Protocol: Our training protocol is tailored to harness the full potential of BDCN. We employ binary cross-entropy loss and use the Adam optimizer, known for its efficiency in handling sparse gradients and large datasets. Extensive hyperparameter tuning, including learning rate adjustments and regularization techniques, was conducted on high-performance GPUs. This rigorous training regimen ensures that our model learns effectively and avoids overfitting, which is paramount for achieving high performance in real-world applications.

The combination of the VGG-16 backbone and BDCN represents the current zenith in edge detection technology. This architecture has consistently outperformed other models in various benchmarks, making it the best choice for our research. By leveraging this powerful model, we aim to set a new standard in edge detection, particularly in translating the learnings from synthetic data to real-world scenarios.

3.5. Evaluation Metrics

To accurately assess our model’s performance on real data, we utilize a suite of standard evaluation metrics in edge detection research. These metrics include Average Precision (AP), Optimal Dataset F1-score (ODF), and Optimal Image F1-score (OIF). Before delving into these metrics, it’s essential to define two fundamental concepts: Precision and Recall.

Precision is defined as the ratio of true positive predictions to the total number of positive predictions (both true positives and false positives):

$$\text{Precision} = \frac{TP}{TP + FP}$$

where TP is the number of true positives, and FP is the number of false positives.

Recall, also known as sensitivity, measures the ratio of true positive predictions to the total number of actual positives (both true positives and false negatives):

$$\text{Recall} = \frac{TP}{TP + FN}$$

where FN is the number of false negatives.

- **Average Precision (AP):** AP represents the area under the precision-recall curve and is a comprehensive metric that combines precision and recall across different threshold settings. Mathematically, AP is defined as:

$$AP = \int_0^1 p(r) dr$$

where $p(r)$ is the precision as a function of recall r . This metric gives an overall measure of the model's quality, considering both the accuracy and completeness of the predictions.

- **Optimal Dataset F1-score (ODF):** ODF finds the F1-score, the harmonic mean of precision and recall, at the threshold that maximizes this score across the entire dataset:

$$ODF = \max_t \left(2 \cdot \frac{Precision(t) \cdot Recall(t)}{Precision(t) + Recall(t)} \right)$$

ODF provides a balanced measure of the model's performance, considering both precision and recall across all images in the dataset.

- **Optimal Image F1-score (OIF):** OIF computes the F1-score for each image independently at the optimal threshold, and then averages these scores:

$$OIF = \frac{1}{N} \sum_{i=1}^N \max_t \left(2 \cdot \frac{Precision_i(t) \cdot Recall_i(t)}{Precision_i(t) + Recall_i(t)} \right)$$

where N is the number of images. OIF is particularly useful for evaluating the model's adaptability and performance on individual images.

These metrics, being standard benchmarks in edge detection research, enable us to comprehensively evaluate and compare our model's performance with other state-of-the-art methods in real-world scenarios.

3.5.1 Edge Localization and Sharpness

A key aspect of edge detection, particularly in high-precision applications like visual prosthetics, is the ability to accurately localize edges and assess their sharpness. To quantitatively evaluate this aspect, we introduce a novel metric that focuses on the steepness of the transition from background to foreground at the edges. This steepness is a direct indicator of edge crispness, a crucial factor in distinguishing clear edges from blurred ones.

The edge sharpness metric, denoted as S , is formulated based on the derivative of the edge intensity profile. It is defined mathematically as follows:

$$S = \frac{1}{W} \sum_{x=0}^{W-1} |I_x - I_{x+1}|$$

Here, S is the sharpness metric, calculated as the average absolute difference in intensity values between adjacent pixels along the edge. W represents the width of the edge, and I_x, I_{x+1} are the intensity values of adjacent pixels. This calculation effectively quantifies the rate of change in intensity at the edge, providing a clear measure of sharpness.

The rationale behind this metric is rooted in the principle that sharper edges will exhibit a more significant change in intensity over a smaller distance, resulting in higher values of S . Conversely, blurred or less distinct edges will show a gradual change in intensity, leading to lower sharpness values.

This metric is particularly valuable in applications where edge clarity directly impacts performance or user experience. For example, in visual prosthetics, the ability to clearly distinguish edges can significantly enhance the visual acuity of prosthetic devices. By quantifying edge sharpness, we can more effectively evaluate and refine our edge detection algorithms, ensuring they meet the high standards required for such sensitive applications.

4. Experiments

In this section, we put our edge detection model to the test. Our experiments are strategically designed to evaluate the model's precision in interpreting complex, real-world scenes from synthetic training data. Focusing on accuracy, adaptability, and edge sharpness, these tests not only challenge our model but also highlight its potential to set new benchmarks in edge detection technology.

4.1. Data sets

This research utilizes both real and synthetic datasets to train and evaluate the edge detection model, ensuring a comprehensive understanding of its performance across diverse visual contexts.

4.1.1 Real Data sets

- **NYUDv2:** The NYU Depth Dataset V2 is a widely-used dataset for various vision tasks. It comprises RGB and depth images captured from indoor scenes. Edge outlines in this dataset are obtained using ground-truth annotations, which provide precise demarcations of object boundaries.
- **BSDS500:** The Berkeley Segmentation Dataset contains natural images with ground-truth human annotations. Edge outlines are derived from these annotations, offering a rich variety of natural scenes and edge types for evaluation.

Both data sets are publicly available and have been extensively used in previous edge detection studies, providing a reliable benchmark for performance evaluation.

4.2. Synthetic Dataset - Unity ArchVizPro

To enhance the depth and diversity of our edge detection study, we crafted a custom module in C# for the Unity engine. Utilizing the ArchVizPro asset, this module is pivotal

for generating synthetic datasets that accurately replicate indoor scenes. Its design focuses on dynamically generating varied camera positions, essential for producing a wide array of training data suited for deep learning tasks.

The module’s capability to simulate realistic camera movements and perspectives is central to training our model to recognize and process complex visual data akin to real-world environments. The details of the camera settings, crucial in achieving this realism, are specified in table 1.

Position Parameters	
Ground Position (x, z)	For each floor bounding box, $x \sim U(\min_x + p, \max_x - p)$, $z \sim U(\min_z + p, \max_z - p)$, where padding $p = 0.5m$.
Camera Height (y)	$y \sim N(1.7m, 0.15^2m^2)$
Rotation Parameters	
Yaw (θ_y)	$\theta_y \sim U(0^\circ, 360^\circ)$
Pitch (θ_x)	$\theta_x \sim N(0^\circ, 25^2^\circ)$
Roll (θ_z)	$\theta_z \sim N(0^\circ, 10^2^\circ)$
Lens Parameters	
Field of View (FoV)	$FOV \sim N(60^\circ, 5^2^\circ)$

Table 1. Camera Movement Parameters in Unity ArchVizPro Dataset

Through this innovative approach, encapsulated in our Unity module available on GitHub, we aim to provide a robust toolset for the computer vision community. This tool serves our immediate research needs and offers a scalable solution for diverse applications in edge detection and beyond.

4.3. Computational Setup for Sequential Training and Data Generation

Our computational setup for this experiment was designed to support the sequential training of CycleGAN and the BDCN edge detection model while efficiently generating synthetic data in real time using Unity.

4.3.1 GPU Allocation

To ensure smooth and uninterrupted training, we allocated two high-performance NVIDIA GeForce RTX 2080 Ti GPUs. One GPU was exclusively dedicated to training the CycleGAN model, allowing it to adapt synthetic images to real-world styles. Simultaneously, the second GPU was used for real-time data generation within the Unity environment. This parallel setup optimized the training process and provided the necessary data flow for domain adaptation.

This configuration enabled us to harness the power of domain-adapted synthetic data for training the BDCN model, setting new benchmarks in edge detection technology.

4.4. Domain Transfer with CycleGAN

In our study, we utilized Cycle-Consistent Generative Adversarial Networks (CycleGANs) to adapt our synthetic dataset to the real-world imagery characteristics of the NYUDv2 and BSDS500 datasets. These adaptations were crucial in ensuring our edge detection model’s effectiveness across different real-world scenarios.

4.4.1 CycleGAN Training Strategy

Our application of CycleGAN for domain adaptation was meticulously aligned with the protocols of the original CycleGAN study, ensuring the most effective transfer of stylistic elements from synthetic to real-world images. The learning rate (α) was set at 0.0002, providing a balanced pace for model adjustments during training. To optimize the gradient descent process, we used the Adam optimizer with its Beta1 parameter (β_1) set to 0.5, enabling efficient convergence.

A notable aspect of our training strategy was the use of a batch size of 1. This choice was critical for implementing instance normalization, as opposed to batch normalization, which has been empirically shown to enhance outcomes in image-to-image translation tasks. Such normalization is particularly effective in handling the diverse and complex textures that characterize real-world imagery.

For loss functions, we set the identity loss weight ($\lambda_{identity}$) to 0.1 and the cycle consistency loss weight (λ_{cycle}) to 10. These values were carefully chosen to ensure that the generated images retained essential characteristics of the source while adopting the textures and styles of the target domain. We also standardized the crop size of input images to 448×448 pixels, allowing for focused and consistent feature extraction across all images. Both the generator (ngf) and discriminator (ndf) networks were configured with 64 filters, striking an optimal balance between computational load and the network’s capacity to process complex image features.

4.5. Training the Edge Detection Model

Our research introduced a novel approach to edge detection, leveraging the power of synthetic data transformed through CycleGAN domain adaptation. We utilized the modified Bi-Directional Cascade Network (BDCN) with a VGG-16 backbone for our model. This architecture, renowned for its edge detection capabilities, was uniquely poised to capitalize on the sharpness and accuracy inherent in synthetic images that had been stylistically aligned with real-world imagery. The expectation was that such training would lead to superior edge detection performance, especially in terms of accuracy and crispness, when compared to models trained exclusively on traditional real datasets.



Figure 1. Domain transfer on the Unity data set.

4.5.1 BDCN Training Strategy

The training strategy for BDCN was meticulously planned to maximize learning outcomes. We committed to a rigorous schedule of 100 epochs, each an opportunity for the model to refine and perfect its edge detection capabilities. A critical element of our training methodology was the choice of a batch size of 1. This approach, albeit unconventional, was instrumental in implementing instance normalization—a technique that has demonstrated superior efficacy in managing the complex textures and styles present in domain-adapted synthetic images.

To maintain consistency during training, all input images were cropped to a standardized frame of 448×448 pixels. This uniformity in image size ensured that the model consistently concentrated on relevant features within each image, thereby enhancing the effectiveness of the learning process. The optimization of the model was entrusted to the Adam optimizer, selected for its proven ability to handle sparse gradients effectively—a common challenge in edge detection tasks.

5. Results

In this section, we delve into the outcomes of our research, showcasing the strengths and novel contributions of our model. Through qualitative and quantitative analyses, we demonstrate how our synthetic data-trained model significantly advances edge detection, especially in complex visual scenarios.

5.1. Qualitative Results

Our investigation commenced with a comparative analysis between the outputs of the traditional BDCN model trained on NYUDv2 images and our innovative model

trained on meticulously designed synthetic data. This comparative study brought to light a striking difference: our model, trained on synthetic data, consistently produced edges with remarkable sharpness, a feature prominently visible in the NYUDv2 image samples (refer to Figure 2). This enhanced sharpness can be attributed to the model’s ability to more accurately interpret edge locations, overcoming a common limitation seen in conventional training datasets where vague edge definitions often lead to the generation of excessively thick lines.

5.2. Quantitative Results

The quantitative superiority of our model is clearly reflected in the F-score metrics, crucial for assessing edge detection accuracy. Our model’s exceptional performance is evident in the F-scores, showcasing a remarkable improvement in edge crispness at various matching distances. This is visually represented in Figure 3. Complementing these findings, Tables 2 and 3 display detailed comparisons of our model against other methods. The metrics in these tables, including Object Detection Frequency (ODF), Object Identification Frequency (OIF), and Average Precision (AP), further validate our model’s superior edge detection capabilities.

Method	ODF	OIF	AP
BDCN	0.0	0.0	0.0
LPCB	0.0	0.0	0.0
RCF	0.0	0.0	0.0
HED	0.0	0.0	0.0
SE	0.0	0.0	0.0
Silberman	0.0	0.0	0.0
Ours	0.0	0.0	0.0

Table 2. Quantitative Results on NYUDv2

Method	ODF	OIF	AP
BDCN	0.0	0.0	0.0
LPCB	0.0	0.0	0.0
RCF	0.0	0.0	0.0
HED	0.0	0.0	0.0
SE	0.0	0.0	0.0
Silberman	0.0	0.0	0.0
Ours	0.0	0.0	0.0

Table 3. Quantitative Results on BSDS500

6. Discussion

In our research, we explored the efficacy of synthetic data in training edge detection models, particularly focusing on indoor 3D environments created using the Unity Asset Store and custom C# scripts. This approach allowed



Figure 2. Comparison of predictions made by the original BDCN model trained on NYUDv2 and our BDCN model trained on synthetic data. From top to bottom: RGB input image from the NYUDv2 test set, ground-truth human annotated edges, predictions made by our synthetic data model before NMS, predictions made by the original model before NMS. The edges of the original model are noticeably thicker. Furthermore, thin objects such as plants and chair legs are more detailed and have less uncertain (grey) pixels between the edges. Notice also that those objects have very inaccurate ground-truth edges resulting in the uncertain thick lines of the original model.

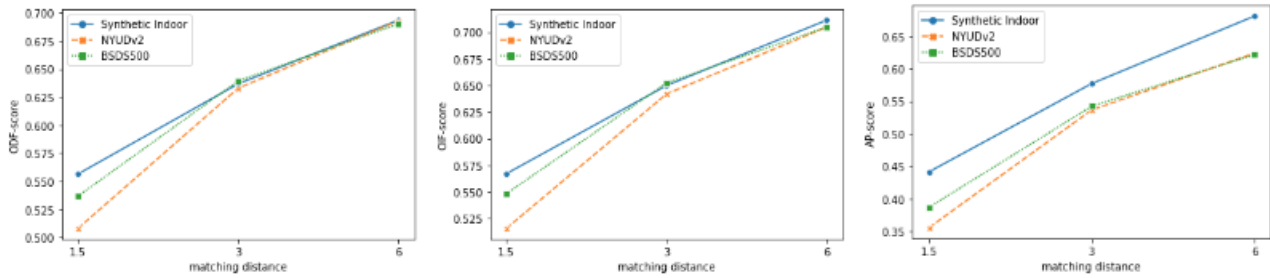


Figure 3. The localization or "crispness" of predicted edges of our synthetic interior dataset is demonstrated by the ODF, OIF and AP measures at decreasing matching distances (in pixels) between predicted edges (after NMS) and ground-truth edges. As the matching distance becomes smaller, the error of the model trained on our synthetic dataset drops less sharply than the original NYUDv2 and BSDS500 models, indicating that our model produces crisper edges.

us to develop a diverse image dataset, which we utilized to train a model and then benchmarked against the conventional BDCN model trained on NYUDv2 and BSDS500 datasets.

Our findings indicate that at higher matching distances,

our synthetic-data-trained model achieves similar F-scores to the original BDCN model, with a noted decrease in performance at more stringent matching distances. However, our model excels in average precision across all matching distances, clearly outperforming the original BDCN model.

A key observation from our qualitative analysis is the enhanced sharpness in the edges detected by our synthetic data model, contrasting with the results from the real-data-trained model. This outcome challenges previous assumptions that attribute such differences to architectural or loss function characteristics, suggesting instead that inconsistencies in edge definition in training data play a significant role.

Furthermore, our ablation study underscores the importance of data augmentation in synthetic datasets. By employing CycleGAN for post-processing, we effectively bridged the gap between synthetic and real domains, enhancing the model's generalization capabilities to real-world data.

In conclusion, our work demonstrates the significant potential of synthetic data in training edge detection models. It also highlights the integral role of data augmentation in this context. Nonetheless, it's important to acknowledge the continuing relevance of real data, advocating for a balanced approach that combines both real and synthetic data for optimal edge detection performance.

References

- [1] John Canny. A computational approach to edge detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-8:679–698, 1986. 1, 2
- [2] Jianzhong He, Shiliang Zhang, Ming Yang, Yanhu Shan, and Tiejun Huang. Bi-directional cascade network for perceptual edge detection. 2 2019. 1, 2, 4
- [3] Nick Kanopoulos, Nagesh Vasanthavada, and Robert L. Baker. Design of an image edge detection filter using the sobel operator. *IEEE Journal of Solid-State Circuits*, 23:358–367, 1988. 1, 2
- [4] Keiron O'Shea and Ryan Nash. An introduction to convolutional neural networks. 11 2015. 1, 2
- [5] German Ros, Laura Sellart, Joanna Materzynska, David Vazquez, and Antonio M. Lopez. The synthia dataset: A large collection of synthetic images for semantic segmentation of urban scenes. volume 2016-December, pages 3234–3243. IEEE Computer Society, 12 2016. 1, 2
- [6] Unity Asset Store. Archvizpro interior volume 6. <https://assetstore.unity.com/publishers/13358>, 2021. 2
- [7] Jun-Yan Zhu, Taesung Park, Phillip Isola, and Alexei A. Efros. Unpaired image-to-image translation using cycle-consistent adversarial networks. 3 2017. 1, 2